DM587

Scientific Programming

Lecture 01: Python foundations

Sai Ganesh Nagarajan

University of Southern Denmark

19 October 2026

Course logistics

Course webpage

https://sgnagarajan1.github.io/dm587/

Lecturer

Sai Ganesh Nagarajan

Email: sgnagarajan@imada.sdu.dk

Teaching assistant

Krishna Narayanan

Email: nanakrish@imada.sdu.dk

Reference for this lecture

ACME Labs

- Intro to Python
- Intro to NumPy

Tools and setup

- Python documentation
- VS Code: Python setup
- Jupyter: installation

Main readings

Programming

ACME Labs — Python Essentials
Labs for *Foundations of Applied Mathematics*.

Linear algebra

Stephen Boyd and Lieven Vandenberghe (2018).
Introduction to Applied Linear Algebra: Vectors, Matrices, and Least Squares. Cambridge University Press.

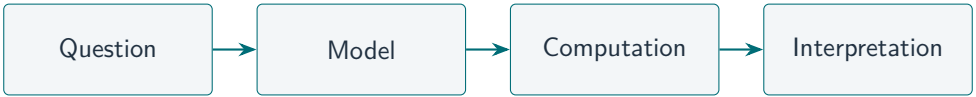
Book website and freely available text.

Today's Class

- Set up Python and run a small calculation.
- Represent measurements with suitable data structures.
- Write functions that use conditions and loops.
- Recognise Python idiosyncrasies that make code clearer and avoid unnecessary work.

By the end: explain how data, functions and conditions form a small program.

Scientific programming in context



- **Question:** What is the positive number whose square is 2?
  Model: Solve $f(x) = x^2 - 2 = 0$ on $[1, 2]$.
  Computation: Repeatedly bisect the interval to obtain $x \approx 1.4142$.
  Interpretation: An approximation to $\sqrt{2}$; check that $x^2 - 2$ is close to zero.

- **Question:** What is the area under $y = x^2$ between 0 and 1?
  Model: Evaluate $I = \int_0^1 x^2 \, dx$.
  Computation: Sum 100 equal-width trapezoids to obtain $I \approx 0.33335$.
  Interpretation: Close to the exact value $1/3$; refine the mesh to check the error decreases.

The wider course

Linear algebra is a central pillar

Vectors and matrices connect mathematical models to computation:
linear systems, least squares, eigenvalues and dimension reduction.

Programming foundations

- Python and numerical arrays
- Working with data
- Checking and explaining results

Scientific applications

- Fitting models to observations
- Graphs and ranking
- Dimension reduction and optimisation

Linear algebra is Central

Physics

Find equilibrium configurations and predict motion:

$$K\mathbf{u} = \mathbf{f}, \qquad \dot{\mathbf{z}} = A\mathbf{z}.$$

Linear systems determine equilibria; eigenvalues describe growth, decay and oscillation in linear dynamics.

AI and machine learning

Fit a model to many observations:

$$\min_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|^2$$

A neural-network layer also starts with a matrix operation, $W\mathbf{x} + \mathbf{b}$.

Mathematics

Represent an unweighted graph by its adjacency matrix A.

$$A, \qquad L = D - A$$

$(A^k)_{ij}$ counts length-k walks from i to j.

For an undirected graph, the Laplacian L, with degree matrix D, connects eigenvalues to connectivity.

The representation determines the computational cost

Memory: store the structure you need

A dense $n \times n$ matrix stores n^2 values. A tridiagonal matrix has at most $3n - 2$ nonzero entries; sparse formats store these values and their locations.

Efficiency: exploit arrays and matrix structure

NumPy array operations use optimised numerical routines. Sparse or structured algorithms can avoid work on entries known to be zero.

Algorithms: compute the solution directly

To solve $A\mathbf{x} = \mathbf{b}$, use a linear-system solver instead of explicitly forming A^{-1} and multiplying by $\mathbf{b}$.

Break a problem into pieces you can explain

- Start with the required inputs and outputs, then divide the task into manageable steps.
- Give each step one clear job; check it on a small example.
- Identify reusable calculations and express them as functions with clear inputs and return values.
- Combine the pieces and check that the whole program solves the original problem.

Programming develops a complementary skill to mathematical reasoning: designing precise, executable steps and organising them into reliable, reusable parts.

A repository keeps your project and its history

What is a repository?

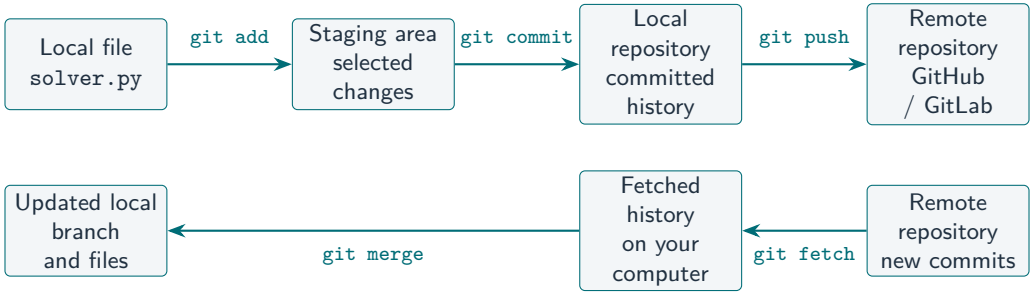
A project tracked by Git: its files together with a recorded history of versions, called *commits*.

- **Compare:** see what changed between a working version and a new experiment.
- **Recover:** retrieve an earlier committed version when a change goes wrong.
- **Collaborate:** combine contributions and keep track of who changed what.

Git manages the history on your computer. GitHub or GitLab can host a remote copy for sharing and keeping committed work elsewhere.

Example: keep one project history rather than files named `solver_final_v3.py`.

The Git workflow: local files and a remote repository



First time: `git clone <repository-url>` creates a local copy of an existing repository and its files.

Bring changes back: fetch downloads history; merge integrates it into your branch. `git pull` combines fetching with integration (merge or rebase, depending on configuration).

If edits conflict, resolve them and test the combined code before continuing.

Git basics

1. Save your files and check that the program runs.
2. Select the files you want to include: `git add`.
3. Record your changes: `git commit`. Use a short message, such as “Add factorial function”.
4. Upload your commits to GitHub: `git push`.

AI usage and learning

Pros

- Can explain syntax, error messages and unfamiliar code.
- Can suggest examples, tests and alternative approaches.
- Can help explore a problem when you evaluate the suggestions yourself.

Cons

- Confident answers can contain mathematical or programming errors.
- Generated code may overlook assumptions and boundary cases.
- We bypass the process of learning and thinking deeply about problems and solutions.

Plagiarism checks: Will be conducted between all pairs of students, thus if two students blindly submit answers from AI, it is likely that this might be flagged.

Takeaway on AI usage

Use this opportunity to explore, make mistakes and build strong foundations. With that understanding, AI can help you go further.

Getting ready

A place to run Python

An interpreter, an editor and a working environment

Section reading

ACME Labs: Intro to Python

Python on Windows; macOS; virtual environments

VS Code tutorial; choosing an environment

IPython; Jupyter; Colab

Different tools, different jobs

Tool	Its role
Terminal	Launch programs and move between folders.
Python / IPython	Execute Python interactively.
VS Code	Edit and save Python files; includes a terminal.
Notebook	Combine code cells, results and explanations.

We will demonstrate the shell first, then run a file written in an editor.

A working setup

- **Run code:** supported Python 3, with IPython for interactive work.
- **Write files:** VS Code or another text editor.
- **Optional notebook interface:** JupyterLab or a browser service.
- **Optional distribution:** Anaconda bundles Python, scientific packages and a graphical package manager.
- **Optional scientific IDE:** Spyder combines an editor, IPython console and variable explorer. It is also available through Anaconda.

VS Code is an editor. It does not replace the Python interpreter.

Python Essentials | Part 1

A working Python session

Expressions, arithmetic and logical questions

Section reading

ACME Labs: Intro to Python — arithmetic, types and comparisons

ACME Labs: The Standard Library

Python documentation: floating-point arithmetic

Simple Calculations

```
print(3 ** 2 + 2 * 5)
print(13 % 3)
print((3 + 2) * 5)
```

OUTPUT

```
19
1
25
```

Parentheses make the intended calculation explicit.

Division and remainders

```
print(15 / 4)      # division: floating-point quotient
print(15 // 4)     # floor division: round down
print(15 % 4)      # remainder after floor division
print(15.0 // 4)   # a float input gives a float result
```

OUTPUT

```
3.75
3
3
3.0
```

$15 = 4 \times 3 + 3$: floor division gives the quotient; % gives the remainder.

Round down means towards $-\infty$: `-15 // 4` is -4, not -3.

Imports

A **module** groups reusable code, such as functions and constants.

```
import math
print(math.sqrt(9))

from math import sqrt
print(sqrt(16))
```

OUTPUT

```
3.0
4.0
```

- `import math`: use the module name, then a dot: `math.sqrt(...)`.
- `from math import sqrt`: use the selected name directly: `sqrt(...)`.
- `import numpy as np` gives NumPy the short name `np`; we will use it later.

Import: use a module in your code. Install: add a package to your environment.

The standard library: tools that come with Python

Selected modules from ACME's *The Standard Library* lab:

Module	What it provides
`math`	Mathematical functions and constants
`cmath`	Mathematical functions for complex numbers
`itertools`	Iteration tools, including combinations and permutations
`random`	Pseudorandom sampling and shuffling
`string`	Useful character groups, such as letters and digits
`sys`	Access to interpreter settings and command-line arguments
`time`	Time-related tools, including timing code

These modules need no separate installation. NumPy, SciPy and pandas are external packages.

Built-ins such as `len` and `sum` need no import.

We will return to individual modules when needed.

Mathematical Example

```
import math

radius = 2.0
area = math.pi * radius ** 2
print(round(area, 3))
print(math.sqrt(9))
```

```
OUTPUT
12.566
3.0
```

`math` is part of Python's standard library; no package installation is needed.

Comparisons produce Boolean values

```
print(3 > 2.99)
print(1.0 <= 1 or 2 > 3)
print(7 == 7 and not 4 < 4)
```

OUTPUT

```
True
True
True
```

= assigns a value. == compares values.

Floating-point equality

```
x = 0.1 + 0.2
print(x == 0.3)
```

Try it

True or False? Why?

Floating-point comparisons

```
import math
x = 0.1 + 0.2
print(x)
print(x == 0.3)
print(math.isclose(x, 0.3, rel_tol=1e-9, abs_tol=1e-12))
```

OUTPUT

```
0.30000000000000004
False
True
```

- Many decimals have no exact finite binary representation; rounding makes the stored sum differ from `0.3`.
- `math.isclose` compares with a tolerance: `rel_tol` scales with magnitude; `abs_tol` is useful near zero.
- Choose tolerances for the problem. For arrays, use `np.isclose` or `np.allclose`.

The same calculation with different float types

```
import numpy as np
dtype = np.float32    # Repeat with np.float16 and np.float64.
a, b, c = dtype("0.1"), dtype("0.2"), dtype("0.3")
print(a + b == c)
```

Results below show stored values with extra digits:

Type	a + b	c	Equal?
float16	0.2998046875	0.300048828125	False
float32	0.30000001192092896	0.30000001192092896	True
float64	0.30000000000000004	0.29999999999999999	False

Python's ordinary `float` behaves like `float64` here.

For `float32`, the sum and c round to the same approximation. This is not exact arithmetic or greater accuracy; compare approximate results with tolerances.

Boolean Operators

```
width, height = 3, 0
valid = width >= 0 and height >= 0
flat = width == 0 or height == 0
print(valid, flat, not valid)
```

OUTPUT

```
True True False
```

- `and`: both conditions must hold.
- `or`: at least one condition must hold.
- `not`: reverses a Boolean value.

Python Essentials | Part 2

Names, values and data structures

Choosing a representation for your data

Section reading

ACME Labs: Intro to Python — names and data structures

ACME Labs: Intro to NumPy; Python Tips for Math Students

Python data structures; NumPy broadcasting; copies and views

The representation depends on the question

What do you want to store or do?	Starting choice
Keep scores $12, 18, 15$ in order, then add another score	list
Store a point's coordinates $(2, 3)$ as one fixed pair	tuple
Find a student's score from their name: Alice $\mapsto 18$	dictionary
Keep only distinct values: $2, 2, 3, 3, 5$ becomes $\{2, 3, 5\}$	set
Add vectors entry by entry: $(1, 2) + (3, 4) = (4, 6)$	NumPy array

For each choice: a small example, followed by the Python behaviour to watch for.

Assignment

```
x = 12
y = 2 * 6
print(x == y)
x, y = 2, 4
print(x == y)
```

OUTPUT

```
True
False
```

Assignment evaluates the right-hand side, then binds the names on the left.

Runtime Typing

```
count = 3              # int
measurement = 3.0      # float
label = "sample A"    # str
accepted = True        # bool
print(type(count), type(measurement))
```

OUTPUT

```
<class 'int'> <class 'float'>
```

Python determines types at runtime. Use descriptive names and consistent meanings.

Input, Numbers and Text

```
raw = input("Temperature in C: ")
value = float(raw)
print(value + 1)
print(f"Reading: {value:.1f} C")
```

OUTPUT

```
Temperature in C: 3.5
4.5
Reading:  3.5 C
```

Example run: the user types `3.5` and presses Enter.

`input()` returns text; convert it when you need a number.

Conversion can fail if the text is not a valid number.

Lists

```
readings = [18.0, 20.5, 21.0, 19.5]
print(readings[0], readings[-1])
print(readings[1:3])
```

OUTPUT

```
18.0 19.5
[20.5, 21.0]
```



Slicing

```
my_string = "Hello world!"
print(my_string[:5])
print(my_string[6:])
print(my_string[::2])
```

OUTPUT

```
Hello
world!
Hlowrd
```

`sequence[start:stop:step]` also works for lists.

Assignment and copying

```
a = [1, 2]
b = a
b[0] = 99
print(a)

b = [7, 8]
print(a, b)

c = a.copy()
c[1] = 0
print(a, c)
```

Try it

Which changes affect `a`? Predict the three output lines.

Assignment and copying

```
a = [1, 2]
b = a
b[0] = 99
print(a)

b = [7, 8]
print(a, b)

c = a.copy()
c[1] = 0
print(a, c)
```

OUTPUT

```
[99, 2]
[99, 2] [7, 8]
[99, 2] [99, 0]
```

1. Change the shared list

`b = a` gives the same list two names. Changing `b[0]` is visible through `a`.

2. Reassign the name

`b = [7, 8]` makes `b` refer to a new list. It does not change the list named by `a`.

3. Change a copied list

`c = a.copy()` creates a new list. Changing an entry in this flat list leaves `a` unchanged.

Sorting

```
values = [3, 1, 2]
ordered = sorted(values)
print(values, ordered)
result = values.sort()
print(values, result)
```

OUTPUT

```
[3, 1, 2] [1, 2, 3]
[1, 2, 3] None
```

`sorted(...)` builds a new list; `list.sort()` changes the existing list.

Do not write `values = values.sort()`: the name would become `None`.

Tuples

```
def arithmetic(a, b):
    return a - b, a * b

both = arithmetic(5, 2)
print(both)
x, y = both
print(x, y)
```

OUTPUT

```
(3, 10)
3 10
```

Tuple entries cannot be reassigned. Assignment can unpack the entries into names.

Dictionaries

```
subjects = {"business": 4121, "math": 2061}
print(subjects["math"])
subjects["science"] = 6284
print(list(subjects))
```

OUTPUT

```
2061
['business', 'math', 'science']
```

Keys are unique and hashable. Modern Python preserves insertion order, unlike the description in this booklet.

Sets

```
members = {"John", "John", "Jane", "Bob"}
print(len(members))
print("Jane" in members)
print(members.intersection({"Jane", "Ian"}))
```

OUTPUT

```
3
True
{'Jane'}
```

A set keeps unique entries. Its printed order is not guaranteed.

NumPy arrays

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
print(x * 2)
print(x ** 2)
print(x.shape)
```

OUTPUT

```
[2.  4.  6.]
[1.  4.  9.]
(3,)
```

For a list, `[1, 2, 3] * 2` repeats entries instead.

Vectorization

```
x = np.array([1.0, 2.0, 3.0])
y = np.empty_like(x)
for i in range(len(x)):
    y[i] = 2 * x[i] + 1

# Same calculation, expressed as array operations.
y = 2 * x + 1
print(y)
```

OUTPUT

```
[3.  5.  7.]
```

For large numeric arrays, NumPy usually avoids the overhead of a Python loop.
Both versions are linear in the number of entries. Array expressions can create temporary arrays; use loops when steps depend on earlier results.

Array shapes

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.shape)
print(a[1, 0])
print(a[:, 1])
```

OUTPUT

```
(2, 3)
4
[2 5]
```

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

`a[row, column]`

Indices begin at zero.

Data structures

Choose a suitable structure for each task. What feature matters?

1. Keep successive approximations $1, 1.5, 1.4167, \ldots$ in order, adding a new value after each step.
2. Store a fixed interval's two endpoints, $a = 0$ and $b = 1$, as one pair.
3. Collect distinct roots from $2, -1, 2, 3, -1$, ignoring their order.
4. Look up a polynomial's coefficient by its power: $3x^2 - 2x + 5$.
5. Store a numerical matrix $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and multiply it by a vector.

Try it

List, tuple, set, dictionary or NumPy array?

Data structures: answers

Task	Suggested structure	Key property
Approximations	`[1, 1.5, 1.4167]`	Ordered; append new values.
Interval	`(0, 1)`	Fixed pair; entries cannot be reassigned.
Distinct roots	`{-1, 2, 3}`	Unique values; order irrelevant.
Coefficients	`{2: 3, 1: -2, 0: 5}`	Power $\mapsto$ coefficient lookup.
Matrix	`np.array([[1, 2], [3, 4]])`	Numerical operations; `A @ v`.

Other representations can work; choose according to the operations and constraints.

Python Essentials | Part 3

Conditions

Conditions, indentation and Python idioms

Section reading

ACME Labs: Intro to Python — conditions; Python Tips for Math Students

Python documentation: control flow; comparisons and Boolean expressions

Conditionals

```
x = -2
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")
```

OUTPUT

```
negative
```

- Conditions are checked in order; only the first matching branch runs.
- Python uses a colon and indentation to define each block; use four spaces.
- Separate `if` statements are independent: more than one may run.

Pythonic conditions

```
x = 5
print(0 <= x < 10)

values = []
print(bool(values))
if values and values[0] > 0:
    print("First value is positive")
```

OUTPUT

```
True
False
```

- **Chain comparisons:** `0 <= x < 10` expresses the interval directly.
- **Use truth values:** an empty list is false; `if values:` checks whether it is non-empty.
- **Short-circuit:** `and` skips the second test when the first is false, so the empty list is never indexed here.

The list truth-value idiom does not apply to multi-element NumPy arrays; use explicit array tests.

Python Essentials | Part 4

Functions

Inputs, results and small functions

Section reading

ACME Labs: Intro to Python — functions; Python Tips for Math Students

Python functions and default arguments; scope and namespaces

ACME Labs: Code Quality; Python documentation: type hints

Functions

```
def area(width, height=1):
    return width * height

print(area(2, 5))
print(area(2))
```

OUTPUT

```
10
2
```

- **Indentation:** the four spaces before `return` mark it as part of the function body.
- **Definition and call:** `def` creates the function; `area(2, 5)` runs its body.
- **Default:** `height=1` is used when the caller omits `height`.

Type hints

Python is **dynamically typed**: values have types, but names need no fixed type declaration.

```
def rectangle_area(width: float, height: float) -> float:
    return width * height

print(rectangle_area(2.0, 5.0))
```

```
OUTPUT
10.0
```

- `width: float` describes an expected input; `-> float` describes the return value.
- Prefer hints on reusable function inputs and outputs. Add variable hints when they clarify intent; obvious local types can be inferred.
- Hints help readers, editors and type checkers. They do not convert values, enforce types at runtime or replace input validation and tests.

Default arguments

```
def area(width, height=1):
    return width * height
print(area(3))
print(area(3, 4))
print(area(height=4, width=3))
```

OUTPUT

```
3
12
12
```

- A default is used only when that argument is omitted.
- Required parameters precede defaults: `width, height=1`.

Positional and keyword arguments

For `def area(width, height=1):`

Positional arguments come first.

```
area(3, height=4)   # valid: returns 12
area(width=3, 4)    # SyntaxError: positional after keyword
```

Keywords may be reordered; each parameter gets one value.

```
area(height=4, width=3)  # valid: returns 12
area(3, width=4)         # TypeError: width supplied twice
```

A positional argument is matched by position; a keyword argument is matched by name. The invalid calls are separate examples to try individually.

Default values

```
scale = 2
def multiply(x, factor=scale):
    return x * factor
scale = 10
print(multiply(3))
print(multiply(3, scale))
```

OUTPUT

```
6
30
```

- Defaults are evaluated once, when `def` executes.
- Changing `scale` later does not change the stored default `factor=2`.
- Prefer numbers, strings or `None` as defaults.

Mutable defaults

Shared default list

```
def record(value, history=[]):
    history.append(value)
    return history

print(record(1))
print(record(2))
```

OUTPUT

```
[1]
[1, 2]
```

The default list is created once. Both calls add to the same list.

Fresh list per call

```
def record(value, history=None):
    if history is None:
        history = []
    history.append(value)
    return history

print(record(1))
print(record(2))
```

OUTPUT

```
[1]
[2]
```

When no list is supplied, each call creates a new one inside the function.

Print and return

```
def oops(i):
    print(i + 1)

def increment(i):
    return i + 1

x = oops(1999)
y = increment(1999)
print(x, y)
```

Try it

Predict every output line. What values are stored in `x` and `y`?

Print and return

```
def oops(i):
    print(i + 1)

def increment(i):
    return i + 1

x = oops(1999)
y = increment(1999)
print(x, y)
```

OUTPUT

```
2000
None 2000
```

Printing a value does not return it to the caller.

Scope

```
offset = 10

def shift(value):
    offset = 2
    return value + offset

print(shift(5), offset)
```

OUTPUT

```
7 10
```

Prefer passing required values as arguments to relying on global state.

Multiple return values

```
def arithmetic(a, b):
    """Return the difference and product."""
    return a - b, a * b

x, y = arithmetic(5, 2)
print(x, y)
```

OUTPUT

```
3 10
```

The returned pair is a tuple; assignment unpacks it into two names.

Functions as arguments

```
def evaluate_at(function, x):
    return function(x)

def square(x):
    return x ** 2

print(evaluate_at(square, 3))
```

OUTPUT

```
9
```

`square` passes the function; `square(3)` calls it.

Lambda expressions

```
def polynomial(x):
    return 6 * x ** 3 + 4 * x ** 2 - x + 3

f = lambda x: 6 * x ** 3 + 4 * x ** 2 - x + 3
print(polynomial(2), f(2))
```

OUTPUT

```
65 65
```

Use `def` when a function needs documentation or several steps.

Recap

- Types and operators
- Imports and the standard library
- Floating-point comparisons
- Lists, tuples, dictionaries and sets
- Assignment, mutation and copying
- NumPy arrays
- Conditions
- Functions, arguments and scope

Next class: loops, comprehensions, NumPy, objects and classes.

References

Main reference

ACME Labs: labs.acme.byu.edu
Labs for *Foundations of Applied Mathematics*.
Intro to Python; Intro to NumPy

Selected examples are reused or adapted from the ACME lab materials and the earlier *Python Essentials* booklet; additional teaching examples and diagrams have been added. Syntax and setup guidance are updated for Python 3.

Attribution: Dr. J. Humpherys, original work at
github.com/Foundations-of-Applied-Mathematics/Labs.
Creative Commons Attribution 3.0 United States.